

Desenvolvimento de um Gerador de Lances para Xadrez em Rust com Bitboards e Magic Bitboards

Guilherme Borges Pagano
Faculdade de Engenharia Elétrica
Universidade Federal de Uberlândia
Uberlândia, Brasil
guilhermepagano@ufu.br

João Marcos de Oliveira Calixto
Faculdade de Engenharia Elétrica
Universidade Federal de Uberlândia
Uberlândia, Brasil
jm.calixto@ufu.br

Augusto W. F. Veloso da Silveira
Faculdade de Engenharia Elétrica
Universidade Federal de Uberlândia
Uberlândia, Brasil
augustofleury@ufu.br

Resumo — A aplicação de inteligência artificial ao xadrez depende de estruturas de dados e algoritmos capazes de representar o tabuleiro e gerar lances com baixo custo computacional. Este artigo apresenta a ChessLib, uma biblioteca de xadrez implementada em Rust, voltada à representação eficiente do estado do jogo e à geração de lances baseada em bitboards e magic bitboards. A solução explora operações bitwise, tabelas de ataque pré-computadas e uma organização modular orientada à extensibilidade, com ênfase em desempenho e segurança de memória. Além de descrever a arquitetura adotada, o trabalho estabelece um protocolo de avaliação experimental baseado em testes *Perft* e em comparação com bibliotecas de referência do domínio.

Palavras-chave: Rust, Xadrez, Inteligência Artificial, Bitboards, Geração de lances;

I. INTRODUÇÃO

O xadrez ocupa, há décadas, um papel central na pesquisa em inteligência artificial, servindo como ambiente de teste para técnicas de busca, avaliação heurística e, mais recentemente, aprendizado por reforço [1]. Nesse domínio, o desempenho do sistema depende não apenas da qualidade da estratégia de decisão, mas também da eficiência com que o estado do jogo é representado e manipulado.

Marcos históricos como o Deep Blue evidenciaram a força de abordagens baseadas em busca altamente otimizada e conhecimento especializado de domínio [1]. Em seguida, sistemas como o AlphaZero e projetos abertos como o Leela Chess Zero reforçaram a relevância de arquiteturas apoiadas em autojogo e redes neurais profundas [2], [3]. Apesar dessas diferenças na camada de decisão, todos esses sistemas dependem de uma infraestrutura de geração de lances correta e eficiente.

Nesse contexto, este trabalho apresenta o desenvolvimento da ChessLib, uma biblioteca de xadrez implementada em Rust, com foco em eficiência, segurança de memória e organização modular. A biblioteca adota bitboards como estrutura principal de representação do tabuleiro e emprega magic bitboards para otimizar a geração de lances de peças deslizantes, explorando operações bitwise e acesso pré-computado a tabelas de ataque [4], [5].

A proposta insere-se no contexto de bibliotecas de base para *engines* de xadrez e ferramentas correlatas, priorizando uma infraestrutura reutilizável para futuras extensões, como mecanismos de busca, funções de avaliação e integração com agentes de inteligência artificial. Mais especificamente, o trabalho

contribui com: a) a descrição da arquitetura da ChessLib e das decisões de implementação adotadas para representação do estado e geração de lances; b) a sistematização do uso de *bitboards* e *magic bitboards* em uma biblioteca escrita em Rust com foco em extensibilidade; e c) uma avaliação experimental baseada em *Perft* e em comparação com implementações de referência, de modo a situar a ChessLib quanto ao custo computacional da geração de lances.

II. TRABALHOS RELACIONADOS

O desenvolvimento de software para xadrez pode ser analisado, de forma geral, em duas frentes complementares. A primeira corresponde aos *engines* completos, concebidos para selecionar lances e disputar partidas de forma autônoma. A segunda reúne bibliotecas de lógica de xadrez, voltadas à representação do estado do jogo, à aplicação de regras e à geração de lances.

A. Engines de Xadrez de Alta Performance

Entre as *engines* open-source contemporâneas, o Stockfish destaca-se como uma das principais referências de desempenho, combinando busca baseada em poda alfa-beta com heurísticas avançadas e avaliação por redes neurais eficientes no formato NNUE [6]. Em paralelo, o Leela Chess Zero (Lc0) representa a abordagem baseada em redes neurais profundas e autojogo, inspirada pela linha introduzida pelo AlphaZero [2], [3]. Esses dois projetos ilustram paradigmas centrais da computação enxadrística atual e reforçam a importância da geração eficiente de lances como componente estrutural de sistemas competitivos. Além disso, a exploração de linguagens modernas para o desenvolvimento de *engines* de alto desempenho não se restringe a implementações em C++ nem a fluxos de experimentação apoiados em Python: trabalhos recentes também investigam a linguagem Go como base para arquiteturas enxadrísticas modulares e competitivas, como exemplifica a *engine* GoFish [7].

B. Bibliotecas de Lógica de Xadrez

Além dos *engines* completos, há bibliotecas especializadas na modelagem do jogo e na manipulação programática do tabuleiro. Essas bibliotecas são particularmente úteis em cenários de prototipagem, ensino, experimentação algorítmica e integração com aplicações maiores, como analisadores, interfaces gráficas, ferramentas de teste e sistemas de inteligência artificial.

No ecossistema Python, a biblioteca “python-chess” tornou-se uma referência amplamente adotada por oferecer representação

de posições, geração de lances, validação de legalidade e manipulação de formatos usuais do domínio enxadrístico [8]. No entanto, por estar inserida em um ambiente interpretado, sua utilização em cenários de alta intensidade computacional tende a apresentar limitações de desempenho quando comparada a implementações em linguagens compiladas.

No ecossistema Rust, a biblioteca “chess” oferece uma referência importante de implementação eficiente para representação do tabuleiro e geração de lances, demonstrando a viabilidade de soluções de alto desempenho nesse ambiente [9].

Nesse contexto, Rust oferece características particularmente relevantes para bibliotecas centrais de xadrez, como desempenho próximo ao de linguagens de sistema, controle explícito de memória e ausência de coletor de lixo, com garantias estáticas de segurança documentadas tanto em sua documentação oficial quanto em literatura acadêmica da ACM sobre a linguagem e seu uso em sistemas [10], [11].

A ChessLib insere-se nesse cenário como uma biblioteca de xadrez em Rust voltada à construção de uma base modular e eficiente para representação do jogo e geração de lances. Sua proposta não é competir diretamente com *engines* completos, mas oferecer uma infraestrutura reutilizável, com ênfase em desempenho e extensibilidade.

III. ARQUITETURA

A ChessLib foi projetada como uma biblioteca modular para representação de posições e geração eficiente de lances em Rust. Sua implementação inspira-se em bibliotecas enxadrísticas de alto desempenho, com ênfase em estrutura compacta de dados, baixo custo de acesso à memória e separação clara entre representação do estado, pré-cálculo de ataques e geração de movimentos [9].

A. Representação do Tabuleiro

A biblioteca adota bitboards como estrutura principal de representação. Nessa abordagem, o tabuleiro é modelado por inteiros de 64 bits, nos quais cada bit corresponde a uma casa. Em vez de estruturas bidimensionais tradicionais, essa modelagem permite representar conjuntos de casas de forma compacta e manipulá-los por meio de operações bitwise executadas diretamente pela CPU [4].



Fig. 1: Little-Endian File Mapping

Na convenção utilizada, baseada em *Little-Endian File Mapping*, a casa “a1” corresponde ao bit menos significativo e “h8” ao mais significativo. A partir dessa organização, a posição pode ser descrita por múltiplos bitboards, tipicamente separados por tipo de peça e cor, além de estruturas agregadas para ocupação total, peças brancas e peças pretas. Essa organização, evidenciada na Fig. 1, simplifica consultas de ocupação, detecção de ataques e aplicação de máscaras sobre regiões específicas do tabuleiro.

B. Geração de Lances

A geração de lances na ChessLib é dividida entre peças de passo e peças deslizantes. Em ambos os casos, a lógica procura deslocar o máximo possível do custo computacional para tabelas pré-calculadas e operações bitwise simples em tempo de execução.

a) Peças de Passo:

Para cavalo e rei, os padrões de ataque dependem apenas da casa de origem; por isso, seus movimentos podem ser pré-computados para as 64 casas e depois filtrados com base na ocupação por peças da mesma cor.

Um cuidado importante nessa etapa é evitar o problema de *wrap-around*, no qual deslocamentos de bits podem produzir ataques inválidos entre bordas opostas do tabuleiro. Esse efeito é evitado mediante máscaras de arquivo aplicadas antes ou depois dos deslocamentos, conforme o padrão de movimento.

No caso dos peões, a geração é mais particular, pois envolve avanço simples, avanço duplo, capturas diagonais, promoção e *en passant*.

O avanço simples pode ser modelado por deslocamento vertical e filtragem pelas casas vazias; o avanço duplo exige, adicionalmente, que a peça esteja na fileira inicial e que não haja bloqueio intermediário. As capturas diagonais também são expressas com deslocamentos bitwise, combinados com máscaras para impedir *wrap-around*. Já promoções e *en passant* dependem de informação adicional de estado, exigindo tratamento específico na lógica de aplicação e validação dos lances.

b) Peças Deslizantes:

Para torres, bispos e damas, entretanto, a geração de lances é substancialmente mais complexa. O conjunto de ataques dessas peças depende da configuração dos bloqueadores presentes ao longo de raios horizontais, verticais ou diagonais. Uma abordagem ingênua, baseada em varrer cada direção em tempo de execução para cada peça, produz custo elevado. Por isso, a ChessLib adota a técnica de *magic bitboards*, que substitui esse processo por indexação em tabelas pré-calculadas [5], [12].

C. Magic Bitboards

O objetivo central dos *magic bitboards* é transformar o problema da geração de ataques de peças deslizantes em um problema de consulta eficiente. Para uma peça deslizante em uma determinada casa, os lances possíveis dependem apenas dos bloqueadores relevantes presentes nos seus raios de ação. Em vez de recalculá-los dinamicamente a cada consulta,

a técnica associa cada configuração relevante de bloqueadores a um índice compacto em uma tabela de ataques pré-computada.

A indexação é feita por uma função de hash baseada em multiplicação e deslocamento:

$$\text{index} = ((\text{occupancy} \& \text{mask}) * \text{magic}) \gg \text{shift}$$

Nessa expressão, “occupancy” representa a ocupação atual do tabuleiro, “mask” seleciona apenas os bloqueadores relevantes para a peça e a casa em questão, “magic” é uma constante de 64 bits escolhida especificamente para essa configuração, e “shift” reduz o resultado a um índice compacto.

O valor produzido é então utilizado para acessar uma tabela cujo conteúdo é um bitboard contendo os ataques válidos correspondentes.

A implementação depende de três elementos principais:

a) Máscara de bloqueadores:

Esse elemento define quais casas realmente influenciam os ataques de uma peça deslizante em determinada casa. Essas casas correspondem aos raios de movimento da peça, excluindo a própria casa de origem, bem como as casas de borda. Essa exclusão é uma otimização importante, pois o estado da casa terminal de um raio é redundante para distinguir conjuntos de ataque, permitindo reduzir o número de combinações relevantes e, conseqüentemente, o tamanho das tabelas.

b) Número Mágico:

O número mágico é uma constante de 64 bits, única para cada casa e tipo de peça (torre/bispo), que foi descoberta através de uma busca por força bruta para satisfazer a propriedade de hashing perfeito para a máscara de bloqueadores dessa casa.

Estes números não são derivados de uma fórmula matemática, mas são encontrados através de um processo de tentativa e erro. Embora esse procedimento seja empírico, a técnica possui validação e lastro na literatura científica de jogos de tabuleiro, tendo sido formalmente proposta e implementada também no contexto do Shogi [13]. A comunidade de programação de xadrez mantém listas dos “melhores mágicos até agora”, que são números que não só funcionam, mas também permitem tabelas de ataque mais compactas [14].

Cada uma das 128 combinações (64 para torres, 64 para bispos) tem o seu próprio número mágico único.

c) Tabela de Ataques:

Por fim, a tabela de ataques armazena, para cada índice válido, o bitboard correspondente ao conjunto de movimentos possíveis. Depois de inicializada, a consulta em tempo de execução torna-se extremamente barata: basta isolar os bloqueadores relevantes, calcular o índice mágico e recuperar o bitboard de ataques armazenado.

A etapa de inicialização consiste justamente em construir essas tabelas e validar números mágicos adequados para bispos e torres em cada uma das 64 casas. Embora essa fase seja relativamente trabalhosa, ela é executada apenas uma vez, deslocando o custo computacional para fora do caminho crítico da geração

de lances. O processo de busca de um número mágico pode ser resumido pelo fluxo apresentado na Fig. 2.

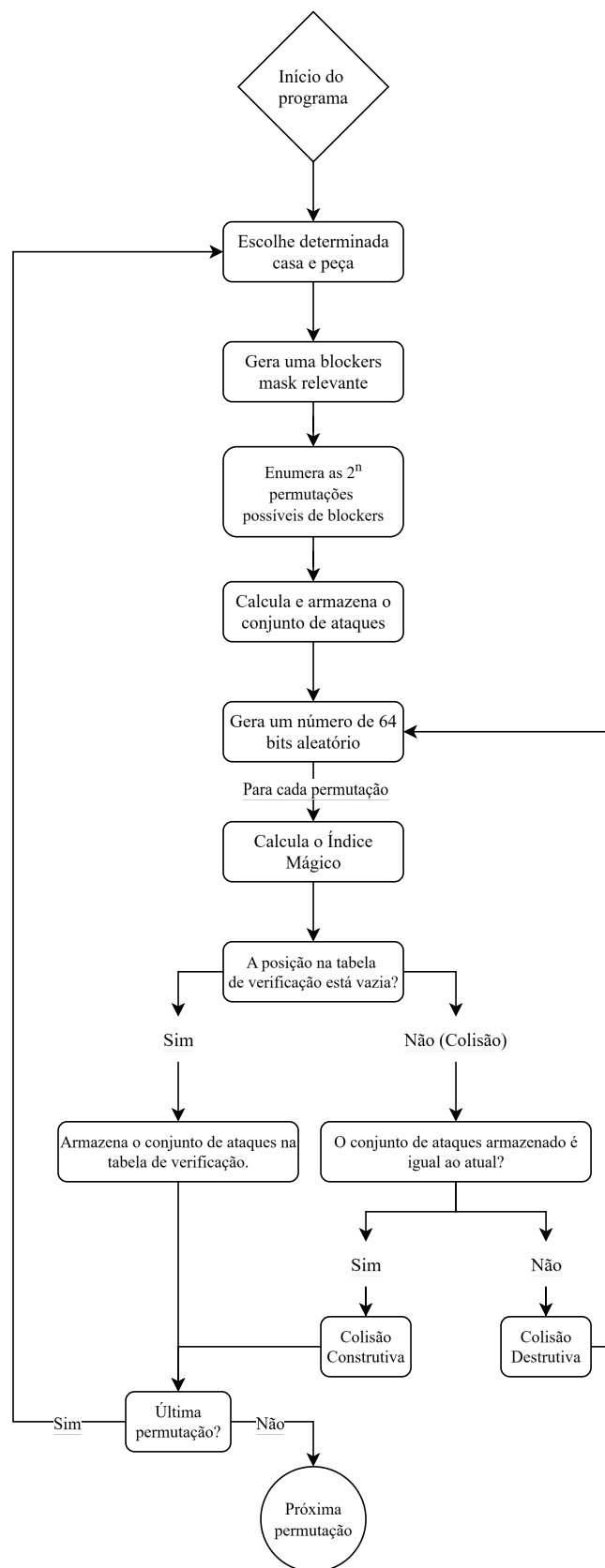


Fig. 2: Geração e validação de um número mágico

Uma vez inicializado, o processo de geração de movimentos em tempo de execução é extremamente eficiente, consistindo em uma sequência linear de operações apresentadas na Fig. 3.

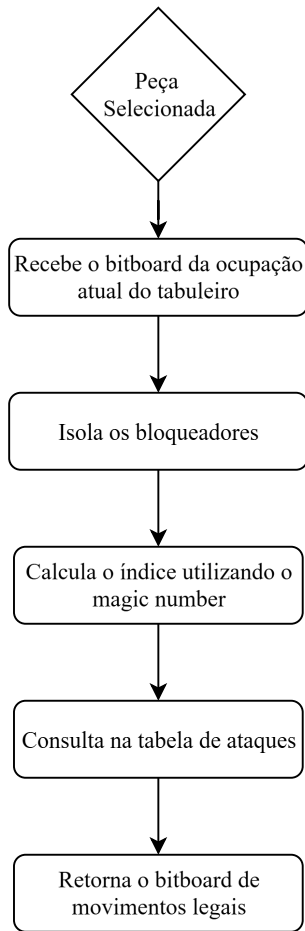


Fig. 3: Consulta de ataques em tempo de execução com magic bitboards

Essa estratégia permite combinar pré-computação, compacidade e eficiência, tornando os magic bitboards a referência consolidada para geração de lances de peças deslizantes em programas de xadrez baseados em bitboards [5].

IV. AVALIAÇÃO EXPERIMENTAL

A avaliação experimental foi estruturada para analisar o desempenho computacional da ChessLib em cenários representativos. Para isso, adotou-se o teste *Perft* (*performance test*), amplamente utilizado em computação enxadrística como base para comparação de desempenho entre implementações [15].

A. Objetivos da Avaliação

O objetivo desta avaliação é medir o custo computacional da geração de lances em uma carga padronizada. Embora o *Perft* não substitua a avaliação de uma *engine* completa, ele constitui um critério adequado para comparar bibliotecas cuja principal responsabilidade é representar o estado do jogo e enumerar movimentos de forma eficiente.

B. Ambiente Experimental

A Tabela I documenta o hardware e o sistema operacional utilizados nos experimentos.

TABELA I: AMBIENTE COMPUTACIONAL UTILIZADO NOS EXPERIMENTOS

Componente	Especificação
CPU	Intel Core i7-12800H @ 4.70GHz
Núcleos/Threads	14 Cores / 20 Threads
RAM	32 GB
Sistema Operacional	Arch Linux x86_64 6.12.42-1-lts

A Tabela II registra as versões das bibliotecas comparadas, uma vez que alterações de implementação entre versões podem impactar diretamente o desempenho.

TABELA II: IMPLEMENTAÇÕES AVALIADAS NOS EXPERIMENTOS

Biblioteca	Versão
ChessLib	1.0
chess	3.2.0
shakmaty	0.30.0
Stockfish via UCI	17
python-chess	1.11.2

A ChessLib foi comparada com cinco implementações de referência:

- “python-chess”, uma biblioteca de xadrez consolidada em Python [8];
- “ChessLib-Simple”, uma variante da ChessLib que não utiliza bitboards, implementada para fins de comparação interna;
- “chess”, uma crate em Rust orientada a alto desempenho [9];
- “shakmaty”, uma biblioteca geral de xadrez em Rust [16];
- “Stockfish”, avaliado por meio de chamadas externas via protocolo UCI [6].

Desse modo, a avaliação busca situar a ChessLib tanto em relação a bibliotecas embutidas no mesmo processo quanto, de forma complementar, em relação ao custo prático de integração com uma *engine* externa.

C. Conjunto de Posições de Teste

Os experimentos foram definidos a partir de um conjunto de posições em notação Forsyth-Edwards (FEN), formato amplamente empregado para representar estados completos de uma partida de xadrez [17]. A seleção das posições teve como objetivo variar a carga de processamento e cobrir aspectos específicos da lógica de geração de lances.

a) Posição Inicial:

A posição inicial foi adotada como referência básica por ser um caso clássico de benchmark em testes *Perft*.

```

rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq -
0 1

```

b) *Teste de capturas*:

Uma posição clássica de *perft* com alta incidência de capturas e tática imediata, útil para estressar verificações de legalidade e a enumeração de respostas forçadas.

```
rnbq1k1r/pp1PbPPP/2p5/8/2B5/8/PPP1NnPP/RNBQK2R w KQ - 1 8
```

c) “*Kiwipete*”:

Uma posição complexa de meio-jogo com muitas possibilidades táticas, incluindo roques, capturas e lances de peão. Testa o desempenho num cenário mais realista e computacionalmente denso.

```
r3k2r/p1ppqpb1/bn2pnp1/3PN3/1p2P3/2N2Q1p/PPPBPPPP/R3K2R w KQkq - 0 1
```

d) *Teste de promoção*:

Uma posição projetada especificamente para testar a lógica de promoção de peões, que pode ser uma fonte de bugs e ineficiências.

```
n1n5/PPPk4/8/8/8/8/4Kppp/5N1N b - - 0 1
```

Na posição inicial, os benchmarks principais foram executados nas profundidades 4 a 7. Nos presets específicos, foram avaliadas as posições captures na profundidade 5, promotions na profundidade 6 e *kiwipete* na profundidade 5. As implementações *chesslib-simple* e *python-chess* foram incluídas apenas na rodada de profundidade 4 da posição inicial, em caráter complementar.

V. RESULTADOS

A. Observações Metodológicas

A execução do Stockfish ocorreu por meio de um processo externo acionado via UCI, de modo que seus tempos incluem inicialização do processo, *handshake* do protocolo e comunicação por *stdin/stdout*; assim, eles não são diretamente comparáveis ao custo *in-process* das crates Rust.

ChessLib-Simple é uma implementação alternativa da ChessLib, que não utiliza, propositalmente, a técnica de *bitboards*.

ChessLib-Simple e Python-Chess só foram incluídos na rodada da profundidade 4 da posição inicial.

B. Posição Inicial

TABELA III: BENCHMARK NA POSIÇÃO INICIAL PARA $d = 4$

Engine	Tempo médio (ms)
ChessLib	1.5
Shakmaty	1.6
Chess	1.2
ChessLib-Simple	23.3
Stockfish via UCI	161.4
Python-Chess	235.0

TABELA IV: BENCHMARK NA POSIÇÃO INICIAL PARA $d = 5$

Engine	Tempo médio (ms)
ChessLib	14.4
Shakmaty	15.8
Chess	11.9
Stockfish via UCI	173.8

TABELA V: BENCHMARK NA POSIÇÃO INICIAL PARA $d = 6$

Engine	Tempo médio (ms)
ChessLib	259.5
Shakmaty	350.2
Chess	241.9
Stockfish via UCI	500.9

TABELA VI: BENCHMARK NA POSIÇÃO INICIAL PARA $d = 7$

Engine	Tempo médio (s)
ChessLib	7.467
Shakmaty	9.318
Chess	6.251
Stockfish via UCI	9.974

C. Posição de capturas ($d = 5$)

TABELA VII: BENCHMARK NA POSIÇÃO DE CAPTURAS PARA $d = 5$

Engine	Tempo médio (ms)
ChessLib	142.5
Shakmaty	249.0
Chess	128.1
Stockfish via UCI	445.1

D. Posição de promoções ($d = 6$)

TABELA VIII: BENCHMARK NA POSIÇÃO DE PROMOÇÕES PARA $d = 6$

Engine	Tempo médio (ms)
ChessLib	195.4
Shakmaty	298.0
Chess	177.2
Stockfish via UCI	570.0

E. Posição “*Kiwipete*” ($d = 5$)

TABELA IX: BENCHMARK NA POSIÇÃO “KIWIPE” PARA $d = 5$

Engine	Tempo médio (ms)
Chess	261.5
Shakmaty	537.3
ChessLib	282.9
Stockfish via UCI	731.7

Para complementar a análise, a Tabela X resume os tempos médios obtidos nos presets “captures”, “promotions” e “kiwipete”.

TABELA X: TEMPOS MÉDIOS EM POSIÇÕES ESPECÍFICAS DE TESTE

Posição de teste	Profundidade	ChessLib	Shakmaty	Chess	Stockfish via UCI
Captures	5	142.5 ms	249.0 ms	128.1 ms	445.1 ms
Promotions	6	195.4 ms	298.0 ms	177.2 ms	570.0 ms
Kiwipete	5	282.9 ms	537.3 ms	261.5 ms	731.7 ms

VI. DISCUSSÃO

Os resultados evidenciam, em primeiro lugar, o impacto decisivo das escolhas de implementação sobre o custo da geração de lances. Na posição inicial com profundidade $d = 4$, a ChessLib executou o *perft* em 1.5 ms, ao passo que a ChessLib-Simple — variante construída propositalmente sem *bitboards* — registrou 23.3 ms, e a biblioteca “python-chess” atingiu 235.0 ms. Esse contraste mostra que o ganho observado não é marginal: a adoção de *bitboards* altera substancialmente o custo da enumeração de lances, enquanto uma implementação em linguagem interpretada tende a ampliar ainda mais essa diferença em cargas intensivas.

Sob outra perspectiva, a proximidade entre a ChessLib e a crate “chess” é particularmente relevante. Embora “chess” tenha apresentado o melhor desempenho bruto, a ChessLib manteve-se próxima dessa referência ao longo de todos os cenários avaliados.

Esse resultado torna-se mais expressivo porque a ChessLib não se limita a uma rotina mínima de geração de lances. A biblioteca também mantém informações incrementais de estado, como *Zobrist hash*, contagem de *half-moves* e estruturas úteis para integração com fluxos associados ao protocolo UCI.

VII. CONCLUSÃO

Este artigo apresentou a ChessLib, uma biblioteca de xadrez em Rust voltada à representação eficiente do tabuleiro e à geração de lances por meio de *bitboards* e *magic bitboards*.

No plano experimental, a ChessLib demonstrou que é possível combinar eficiência, correção estrutural e manutenção de estado adicional em uma mesma biblioteca. Os resultados mostraram que a implementação proposta permaneceu muito próxima da crate “chess”, uma das principais referências de desempenho no ecossistema Rust, ao mesmo tempo em que superou com ampla margem a variante ChessLib-Simple, construída sem *bitboards*, e a biblioteca “python-chess”. Esse comportamento indica que as decisões centrais de projeto adotadas na ChessLib foram efetivas e que o ganho observado decorre de escolhas estruturais de representação, e não apenas de otimizações pontuais.

Desse modo, o trabalho não reivindica a superação da principal referência avaliada em Rust, mas mostra que a ChessLib alcançou um patamar competitivo e tecnicamente robusto,

mesmo incorporando responsabilidades adicionais além da enumeração mínima de lances. Em conjunto, esses resultados sustentam a biblioteca como uma base promissora para extensões futuras, como mecanismos de busca, funções de avaliação e, em particular, a implementação de uma inteligência artificial capaz de explorar essa infraestrutura para seleção autônoma de movimentos.

REFERÊNCIAS

- [1] M. Campbell, A. J. J. Hoane, e F.-h. Hsu, “Deep Blue”, 2002, *Elsevier*.
- [2] D. Silver *et al.*, “Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm”, 2017. Acesso em: 9 de abril de 2026. [Online]. Disponível em: <https://arxiv.org/abs/1712.01815>
- [3] The Leela Chess Zero Team, “Overview”. Acesso em: 9 de abril de 2026. [Online]. Disponível em: <https://lczero.org/dev/overview/>
- [4] Chessprogramming Wiki, “Bitboards”. Acesso em: 9 de abril de 2026. [Online]. Disponível em: <https://www.chessprogramming.org/Bitboards>
- [5] P. Kannan, “Magic Move-Bitboard Generation in Computer Chess”, 2007. Acesso em: 9 de abril de 2026. [Online]. Disponível em: https://pradu.us/old/Nov27_2008/Buzz/research/magic/Bitboards.pdf
- [6] The Stockfish Team, “Stockfish Docs”. Acesso em: 9 de abril de 2026. [Online]. Disponível em: <https://official-stockfish.github.io/docs/stockfish-wiki/Stockfish-FAQ.html>
- [7] F. Jubair, A. Abu Salem, Y. Assaf, e K. A. Darabkh, “GoFish: A customizable open-source chess engine in Go”, 2024, *Elsevier*. Acesso em: 10 de abril de 2026. [Online]. Disponível em: <https://doi.org/10.1016/j.eswa.2024.123855>
- [8] N. Fiekas, “python-chess Documentation”. Acesso em: 9 de abril de 2026. [Online]. Disponível em: <https://python-chess.readthedocs.io/>
- [9] J. Bray, “chess”. Acesso em: 9 de abril de 2026. [Online]. Disponível em: <https://docs.rs/chess/latest/chess/>
- [10] S. Klabnik e C. Nichols, *The Rust Programming Language*. No Starch Press, 2023. Acesso em: 9 de abril de 2026. [Online]. Disponível em: <https://doc.rust-lang.org/book/>
- [11] N. D. Matsakis e F. S. Klock II, “The Rust Language”, 2014, *ACM*. Acesso em: 10 de abril de 2026. [Online]. Disponível em: <https://doi.org/10.1145/2663171.2663188>
- [12] Chessprogramming Wiki, “Magic Bitboards”. Acesso em: 9 de abril de 2026. [Online]. Disponível em: https://www.chessprogramming.org/Magic_Bitboards
- [13] I. Yamamoto, S. Takeuchi, T. Kaneko, e T. Tanaka, “Proposal and Implementation of Magic Bitboards in Shogi”, 2010, *Information Processing Society of Japan (IPSJ)*. Acesso em: 10 de abril de 2026. [Online]. Disponível em: https://ipsj.ixsq.nii.ac.jp/record/71312/files/IPSJ-GPWS_2010007.pdf
- [14] N. Fiekas, “Finding Hash Functions for Bitboard Based Move Generation”, 2018. Acesso em: 9 de abril de 2026. [Online]. Disponível em: <https://backscattering.de/magics2.pdf>
- [15] Chessprogramming Wiki, “Perft”. Acesso em: 9 de abril de 2026. [Online]. Disponível em: <https://www.chessprogramming.org/Perft>
- [16] N. Fiekas, “shakmaty”. Acesso em: 10 de abril de 2026. [Online]. Disponível em: <https://docs.rs/shakmaty/0.30.0/shakmaty/>
- [17] S. J. Edwards, “Portable Game Notation Specification and Implementation Guide”. Acesso em: 9 de abril de 2026. [Online]. Disponível em: <https://www.saremba.de/chessgml/standards/pgn/pgn-complete.htm>